

Step 1: Load the Data

Load both CSVs into Pandas DataFrames

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# load the datasets
cust=pd.read_csv(r"D:\python\python datasets\Customer_Master_Data mini
project_dataset.csv")
tran=pd.read_csv(r"D:\python\python datasets\Customer_Transactions
mimi_project.csv")
```

shape, structure, and preview

```
#Display the first few rows of each dataset
cust.head()
```

	CustomerID	Name	Email	Gender	Age
City \					
0	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54
Delhi					
1	CUST10001	Divit Kohli	mkalita@sarin.com	Female	48
Kolkata					
2	CUST10002	Kiara Behl	apteanay@hotmail.com	Male	75
Kolkata					
3	CUST10003	Vaibhav Sankar	bseshadri@choudhry.info	Male	62
Pune					
4	CUST10004	Shray D'Alia	bdhillon@toor-mall.com	Male	55
Delhi					

	MaritalStatus	NumChildren	JoinDate
0	Divorced	0	2021-02-22
1	Married	0	2023-12-06
2	Widowed	2	2023-08-23
3	Divorced	2	2022-11-17
4	Divorced	0	2022-12-04

```
cust.shape
```

```
(1000, 9)
```

```
cust.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
#   :-----
```

```

---  -----
0  CustomerID      1000 non-null  object
1  Name           1000 non-null  object
2  Email          1000 non-null  object
3  Gender         1000 non-null  object
4  Age            1000 non-null  int64
5  City           1000 non-null  object
6  MaritalStatus  1000 non-null  object
7  NumChildren    1000 non-null  int64
8  JoinDate       1000 non-null  object
dtypes: int64(2), object(7)
memory usage: 70.4+ KB

tran.head()

  CustomerID  TransactionDate  TransactionAmount
0  CUST10771          7/31/23             2383.07
1  CUST10100          3/10/24             497.54
2  CUST10031          2/17/25             536.78
3  CUST10987          7/17/23             314.89
4  CUST10831         12/15/24            2543.19

tran.shape

(23050, 3)

tran.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23050 entries, 0 to 23049
Data columns (total 3 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   CustomerID            23050 non-null  object
1   TransactionDate        23050 non-null  object
2   TransactionAmount      23050 non-null  float64
dtypes: float64(1), object(2)
memory usage: 540.4+ KB

```

Data loading insight:

The dataset holds transaction records for 1,000 customers, giving a robust foundation for customer behavior and revenue analysis

Step 2: Clean the Data

Convert JoinDate and TransactionDate columns to datetime

```
#convert date columns to datetime format
tran['TransactionDate'] =
pd.to_datetime(tran['TransactionDate'],errors='coerce')

C:\Users\user\AppData\Local\Temp\ipykernel_13544\1129678865.py:2:
UserWarning: Could not infer format, so each element will be parsed
individually, falling back to `dateutil`. To ensure parsing is
consistent and as-expected, please specify a format.
    tran['TransactionDate'] =
pd.to_datetime(tran['TransactionDate'],errors='coerce')

#convert date columns to datetime format
cust['JoinDate'] = pd.to_datetime(cust['JoinDate'],errors='coerce')

#JoinDate column converted to datetime format
cust.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   CustomerID      1000 non-null   object
1   Name            1000 non-null   object
2   Email           1000 non-null   object
3   Gender          1000 non-null   object
4   Age             1000 non-null   int64
5   City            1000 non-null   object
6   MaritalStatus   1000 non-null   object
7   NumChildren     1000 non-null   int64
8   JoinDate        1000 non-null   datetime64[ns]
dtypes: datetime64[ns](1), int64(2), object(6)
memory usage: 70.4+ KB

#TransactionDate coulmn converted to datetime format
tran.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23050 entries, 0 to 23049
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   CustomerID      23050 non-null   object
1   TransactionDate  23050 non-null   datetime64[ns]
2   TransactionAmount 23050 non-null   float64
```

```
dtypes: datetime64[ns](1), float64(1), object(1)
memory usage: 540.4+ KB
```

Ensure no nulls or bad types

```
#Detection of missing values in the datasets
cust.isnull().sum()

CustomerID      0
Name            0
Email          0
Gender         0
Age            0
City           0
MaritalStatus  0
NumChildren    0
JoinDate       0
dtype: int64

#Detection of missing values in the transaction dataset
tran.isnull().sum()

CustomerID      0
TransactionDate 0
TransactionAmount 0
dtype: int64

# Handling - Drop row with any missing values
tran.dropna(inplace=True)
cust.dropna(inplace=True)
```

Validate uniqueness of CustomerID in master dataset

```
#Check for duplicate CustomerID in the customer dataset
cust['CustomerID'].duplicated().sum()

np.int64(0)

#Check for duplicate CustomerID in the transaction dataset
tran['CustomerID'].isin(cust['CustomerID']).all()

np.True_

#Check how many unique CustomerID present in transaction dataset
tran['CustomerID'].unique()

array(['CUST10771', 'CUST10100', 'CUST10031', 'CUST10987',
       'CUST10831',
       'CUST10404', 'CUST10488', 'CUST10988', 'CUST10657',
       'CUST10007',
       'CUST10897', 'CUST10300', 'CUST10710', 'CUST10480',
```

'CUST10377',
 'CUST10184', 'CUST10214', 'CUST10892', 'CUST10561',
'CUST10879',
 'CUST10959', 'CUST10204', 'CUST10531', 'CUST10755',
'CUST10805',
 'CUST10547', 'CUST10231', 'CUST10141', 'CUST10443',
'CUST10715',
 'CUST10919', 'CUST10765', 'CUST10203', 'CUST10734',
'CUST10373',
 'CUST10503', 'CUST10233', 'CUST10049', 'CUST10251',
'CUST10850',
 'CUST10117', 'CUST10991', 'CUST10371', 'CUST10068',
'CUST10712',
 'CUST10043', 'CUST10217', 'CUST10442', 'CUST10339',
'CUST10754',
 'CUST10416', 'CUST10460', 'CUST10452', 'CUST10837',
'CUST10471',
 'CUST10901', 'CUST10672', 'CUST10609', 'CUST10763',
'CUST10631',
 'CUST10669', 'CUST10659', 'CUST10459', 'CUST10380',
'CUST10040',
 'CUST10208', 'CUST10932', 'CUST10697', 'CUST10265',
'CUST10133',
 'CUST10846', 'CUST10953', 'CUST10530', 'CUST10776',
'CUST10757',
 'CUST10110', 'CUST10806', 'CUST10778', 'CUST10904',
'CUST10425',
 'CUST10858', 'CUST10335', 'CUST10105', 'CUST10854',
'CUST10005',
 'CUST10890', 'CUST10228', 'CUST10963', 'CUST10735',
'CUST10818',
 'CUST10393', 'CUST10199', 'CUST10310', 'CUST10834',
'CUST10316',
 'CUST10675', 'CUST10797', 'CUST10383', 'CUST10999',
'CUST10928',
 'CUST10238', 'CUST10028', 'CUST10694', 'CUST10912',
'CUST10851',
 'CUST10255', 'CUST10610', 'CUST10284', 'CUST10312',
'CUST10185',
 'CUST10817', 'CUST10681', 'CUST10717', 'CUST10902',
'CUST10015',
 'CUST10356', 'CUST10746', 'CUST10961', 'CUST10158',
'CUST10711',
 'CUST10579', 'CUST10768', 'CUST10398', 'CUST10862',
'CUST10502',
 'CUST10466', 'CUST10798', 'CUST10859', 'CUST10702',
'CUST10115',
 'CUST10994', 'CUST10613', 'CUST10359', 'CUST10063',
'CUST10084',

'CUST10247', 'CUST10175', 'CUST10202', 'CUST10437',
'CUST10143',
'CUST10764', 'CUST10387', 'CUST10410', 'CUST10601',
'CUST10360',
'CUST10038', 'CUST10980', 'CUST10873', 'CUST10560',
'CUST10820',
'CUST10870', 'CUST10847', 'CUST10587', 'CUST10362',
'CUST10294',
'CUST10719', 'CUST10381', 'CUST10761', 'CUST10801',
'CUST10336',
'CUST10946', 'CUST10903', 'CUST10957', 'CUST10011',
'CUST10954',
'CUST10875', 'CUST10396', 'CUST10308', 'CUST10279',
'CUST10059',
'CUST10685', 'CUST10863', 'CUST10779', 'CUST10523',
'CUST10491',
'CUST10283', 'CUST10053', 'CUST10325', 'CUST10721',
'CUST10438',
'CUST10595', 'CUST10905', 'CUST10575', 'CUST10860',
'CUST10524',
'CUST10225', 'CUST10174', 'CUST10042', 'CUST10262',
'CUST10196',
'CUST10322', 'CUST10206', 'CUST10758', 'CUST10114',
'CUST10627',
'CUST10171', 'CUST10430', 'CUST10431', 'CUST10258',
'CUST10621',
'CUST10742', 'CUST10132', 'CUST10095', 'CUST10264',
'CUST10245',
'CUST10984', 'CUST10286', 'CUST10584', 'CUST10895',
'CUST10188',
'CUST10464', 'CUST10840', 'CUST10435', 'CUST10017',
'CUST10148',
'CUST10522', 'CUST10145', 'CUST10421', 'CUST10037',
'CUST10478',
'CUST10519', 'CUST10938', 'CUST10767', 'CUST10507',
'CUST10874',
'CUST10920', 'CUST10981', 'CUST10743', 'CUST10045',
'CUST10108',
'CUST10705', 'CUST10829', 'CUST10750', 'CUST10301',
'CUST10423',
'CUST10474', 'CUST10670', 'CUST10545', 'CUST10887',
'CUST10420',
'CUST10027', 'CUST10880', 'CUST10268', 'CUST10615',
'CUST10298',
'CUST10333', 'CUST10518', 'CUST10029', 'CUST10556',
'CUST10056',
'CUST10223', 'CUST10323', 'CUST10759', 'CUST10099',
'CUST10167',
'CUST10309', 'CUST10422', 'CUST10532', 'CUST10799',

'CUST10985',
 'CUST10146', 'CUST10081', 'CUST10998', 'CUST10536',
'CUST10996',
 'CUST10533', 'CUST10064', 'CUST10436', 'CUST10374',
'CUST10945',
 'CUST10643', 'CUST10848', 'CUST10153', 'CUST10800',
'CUST10415',
 'CUST10626', 'CUST10288', 'CUST10510', 'CUST10997',
'CUST10568',
 'CUST10079', 'CUST10542', 'CUST10138', 'CUST10018',
'CUST10647',
 'CUST10090', 'CUST10990', 'CUST10210', 'CUST10623',
'CUST10713',
 'CUST10372', 'CUST10527', 'CUST10151', 'CUST10787',
'CUST10876',
 'CUST10178', 'CUST10135', 'CUST10434', 'CUST10777',
'CUST10046',
 'CUST10076', 'CUST10364', 'CUST10653', 'CUST10319',
'CUST10701',
 'CUST10257', 'CUST10740', 'CUST10369', 'CUST10378',
'CUST10317',
 'CUST10408', 'CUST10487', 'CUST10448', 'CUST10906',
'CUST10638',
 'CUST10155', 'CUST10489', 'CUST10256', 'CUST10652',
'CUST10983',
 'CUST10591', 'CUST10482', 'CUST10619', 'CUST10543',
'CUST10824',
 'CUST10505', 'CUST10352', 'CUST10506', 'CUST10134',
'CUST10457',
 'CUST10909', 'CUST10663', 'CUST10485', 'CUST10977',
'CUST10865',
 'CUST10154', 'CUST10341', 'CUST10071', 'CUST10699',
'CUST10566',
 'CUST10576', 'CUST10811', 'CUST10640', 'CUST10419',
'CUST10311',
 'CUST10276', 'CUST10226', 'CUST10010', 'CUST10671',
'CUST10366',
 'CUST10082', 'CUST10269', 'CUST10497', 'CUST10736',
'CUST10402',
 'CUST10470', 'CUST10259', 'CUST10499', 'CUST10329',
'CUST10753',
 'CUST10390', 'CUST10124', 'CUST10113', 'CUST10803',
'CUST10908',
 'CUST10914', 'CUST10147', 'CUST10886', 'CUST10943',
'CUST10538',
 'CUST10159', 'CUST10871', 'CUST10062', 'CUST10193',
'CUST10342',
 'CUST10296', 'CUST10446', 'CUST10219', 'CUST10971',
'CUST10589',

'CUST10327', 'CUST10414', 'CUST10252', 'CUST10307',
'CUST10358',
'CUST10885', 'CUST10332', 'CUST10843', 'CUST10123',
'CUST10691',
'CUST10034', 'CUST10841', 'CUST10295', 'CUST10111',
'CUST10617',
'CUST10921', 'CUST10559', 'CUST10504', 'CUST10106',
'CUST10455',
'CUST10941', 'CUST10320', 'CUST10911', 'CUST10948',
'CUST10477',
'CUST10644', 'CUST10240', 'CUST10139', 'CUST10321',
'CUST10496',
'CUST10926', 'CUST10224', 'CUST10716', 'CUST10684',
'CUST10212',
'CUST10629', 'CUST10970', 'CUST10618', 'CUST10465',
'CUST10812',
'CUST10282', 'CUST10687', 'CUST10693', 'CUST10472',
'CUST10481',
'CUST10433', 'CUST10069', 'CUST10964', 'CUST10230',
'CUST10392',
'CUST10558', 'CUST10732', 'CUST10511', 'CUST10338',
'CUST10927',
'CUST10207', 'CUST10070', 'CUST10274', 'CUST10508',
'CUST10498',
'CUST10479', 'CUST10326', 'CUST10476', 'CUST10611',
'CUST10745',
'CUST10660', 'CUST10116', 'CUST10724', 'CUST10585',
'CUST10827',
'CUST10835', 'CUST10913', 'CUST10426', 'CUST10760',
'CUST10935',
'CUST10391', 'CUST10098', 'CUST10570', 'CUST10969',
'CUST10150',
'CUST10003', 'CUST10573', 'CUST10695', 'CUST10632',
'CUST10849',
'CUST10103', 'CUST10083', 'CUST10127', 'CUST10596',
'CUST10572',
'CUST10658', 'CUST10842', 'CUST10868', 'CUST10662',
'CUST10461',
'CUST10955', 'CUST10451', 'CUST10407', 'CUST10637',
'CUST10557',
'CUST10622', 'CUST10136', 'CUST10468', 'CUST10882',
'CUST10340',
'CUST10641', 'CUST10144', 'CUST10271', 'CUST10674',
'CUST10044',
'CUST10096', 'CUST10170', 'CUST10749', 'CUST10036',
'CUST10004',
'CUST10334', 'CUST10346', 'CUST10165', 'CUST10449',
'CUST10633',
'CUST10441', 'CUST10562', 'CUST10411', 'CUST10125',

'CUST10893',
 'CUST10058', 'CUST10989', 'CUST10517', 'CUST10664',
'CUST10399',
 'CUST10966', 'CUST10131', 'CUST10142', 'CUST10582',
'CUST10567',
 'CUST10149', 'CUST10602', 'CUST10242', 'CUST10915',
'CUST10947',
 'CUST10852', 'CUST10972', 'CUST10501', 'CUST10993',
'CUST10648',
 'CUST10000', 'CUST10916', 'CUST10244', 'CUST10727',
'CUST10388',
 'CUST10878', 'CUST10273', 'CUST10958', 'CUST10331',
'CUST10826',
 'CUST10525', 'CUST10795', 'CUST10853', 'CUST10838',
'CUST10263',
 'CUST10581', 'CUST10780', 'CUST10376', 'CUST10918',
'CUST10766',
 'CUST10762', 'CUST10951', 'CUST10093', 'CUST10728',
'CUST10939',
 'CUST10692', 'CUST10385', 'CUST10549', 'CUST10869',
'CUST10330',
 'CUST10781', 'CUST10236', 'CUST10020', 'CUST10078',
'CUST10183',
 'CUST10709', 'CUST10486', 'CUST10077', 'CUST10285',
'CUST10774',
 'CUST10484', 'CUST10191', 'CUST10299', 'CUST10454',
'CUST10152',
 'CUST10201', 'CUST10313', 'CUST10073', 'CUST10968',
'CUST10172',
 'CUST10401', 'CUST10642', 'CUST10816', 'CUST10896',
'CUST10792',
 'CUST10600', 'CUST10786', 'CUST10696', 'CUST10832',
'CUST10976',
 'CUST10807', 'CUST10365', 'CUST10002', 'CUST10809',
'CUST10553',
 'CUST10168', 'CUST10128', 'CUST10965', 'CUST10289',
'CUST10395',
 'CUST10344', 'CUST10351', 'CUST10192', 'CUST10218',
'CUST10006',
 'CUST10291', 'CUST10227', 'CUST10112', 'CUST10182',
'CUST10160',
 'CUST10450', 'CUST10974', 'CUST10328', 'CUST10514',
'CUST10293',
 'CUST10592', 'CUST10130', 'CUST10967', 'CUST10899',
'CUST10630',
 'CUST10534', 'CUST10249', 'CUST10463', 'CUST10235',
'CUST10526',
 'CUST10324', 'CUST10962', 'CUST10686', 'CUST10272',
'CUST10655',

'CUST10429', 'CUST10978', 'CUST10936', 'CUST10379',
'CUST10513',
'CUST10162', 'CUST10035', 'CUST10030', 'CUST10267',
'CUST10253',
'CUST10493', 'CUST10737', 'CUST10739', 'CUST10612',
'CUST10937',
'CUST10345', 'CUST10091', 'CUST10179', 'CUST10804',
'CUST10368',
'CUST10198', 'CUST10458', 'CUST10355', 'CUST10080',
'CUST10097',
'CUST10241', 'CUST10748', 'CUST10535', 'CUST10569',
'CUST10290',
'CUST10828', 'CUST10931', 'CUST10923', 'CUST10216',
'CUST10475',
'CUST10793', 'CUST10855', 'CUST10444', 'CUST10074',
'CUST10157',
'CUST10616', 'CUST10490', 'CUST10120', 'CUST10975',
'CUST10200',
'CUST10676', 'CUST10412', 'CUST10646', 'CUST10950',
'CUST10382',
'CUST10577', 'CUST10726', 'CUST10469', 'CUST10137',
'CUST10521',
'CUST10772', 'CUST10821', 'CUST10121', 'CUST10995',
'CUST10349',
'CUST10839', 'CUST10544', 'CUST10397', 'CUST10403',
'CUST10354',
'CUST10528', 'CUST10406', 'CUST10782', 'CUST10790',
'CUST10731',
'CUST10992', 'CUST10009', 'CUST10541', 'CUST10266',
'CUST10024',
'CUST10625', 'CUST10101', 'CUST10723', 'CUST10413',
'CUST10054',
'CUST10924', 'CUST10730', 'CUST10944', 'CUST10140',
'CUST10770',
'CUST10639', 'CUST10744', 'CUST10047', 'CUST10580',
'CUST10347',
'CUST10960', 'CUST10789', 'CUST10213', 'CUST10883',
'CUST10370',
'CUST10357', 'CUST10447', 'CUST10389', 'CUST10929',
'CUST10867',
'CUST10094', 'CUST10819', 'CUST10428', 'CUST10386',
'CUST10796',
'CUST10075', 'CUST10680', 'CUST10720', 'CUST10014',
'CUST10540',
'CUST10823', 'CUST10825', 'CUST10246', 'CUST10708',
'CUST10229',
'CUST10578', 'CUST10039', 'CUST10186', 'CUST10361',
'CUST10677',
'CUST10864', 'CUST10565', 'CUST10418', 'CUST10275',

'CUST10830',
 'CUST10599', 'CUST10814', 'CUST10891', 'CUST10427',
'CUST10845',
 'CUST10367', 'CUST10866', 'CUST10884', 'CUST10085',
'CUST10032',
 'CUST10250', 'CUST10453', 'CUST10092', 'CUST10197',
'CUST10033',
 'CUST10087', 'CUST10066', 'CUST10051', 'CUST10706',
'CUST10546',
 'CUST10552', 'CUST10682', 'CUST10022', 'CUST10769',
'CUST10949',
 'CUST10315', 'CUST10211', 'CUST10495', 'CUST10065',
'CUST10930',
 'CUST10802', 'CUST10665', 'CUST10678', 'CUST10107',
'CUST10836',
 'CUST10013', 'CUST10400', 'CUST10539', 'CUST10872',
'CUST10318',
 'CUST10516', 'CUST10119', 'CUST10555', 'CUST10603',
'CUST10409',
 'CUST10620', 'CUST10606', 'CUST10894', 'CUST10722',
'CUST10925',
 'CUST10690', 'CUST10220', 'CUST10624', 'CUST10688',
'CUST10783',
 'CUST10239', 'CUST10877', 'CUST10661', 'CUST10205',
'CUST10086',
 'CUST10001', 'CUST10952', 'CUST10933', 'CUST10791',
'CUST10598',
 'CUST10102', 'CUST10982', 'CUST10187', 'CUST10808',
'CUST10248',
 'CUST10586', 'CUST10234', 'CUST10666', 'CUST10222',
'CUST10440',
 'CUST10733', 'CUST10900', 'CUST10126', 'CUST10815',
'CUST10060',
 'CUST10651', 'CUST10520', 'CUST10180', 'CUST10898',
'CUST10679',
 'CUST10194', 'CUST10232', 'CUST10260', 'CUST10698',
'CUST10656',
 'CUST10881', 'CUST10209', 'CUST10055', 'CUST10292',
'CUST10237',
 'CUST10303', 'CUST10462', 'CUST10752', 'CUST10337',
'CUST10215',
 'CUST10512', 'CUST10889', 'CUST10012', 'CUST10529',
'CUST10494',
 'CUST10057', 'CUST10773', 'CUST10348', 'CUST10287',
'CUST10509',
 'CUST10314', 'CUST10305', 'CUST10813', 'CUST10683',
'CUST10718',
 'CUST10161', 'CUST10917', 'CUST10041', 'CUST10353',
'CUST10634',

'CUST10424', 'CUST10306', 'CUST10922', 'CUST10016',
'CUST10574',
'CUST10195', 'CUST10605', 'CUST10822', 'CUST10714',
'CUST10026',
'CUST10483', 'CUST10907', 'CUST10439', 'CUST10025',
'CUST10304',
'CUST10261', 'CUST10956', 'CUST10467', 'CUST10166',
'CUST10636',
'CUST10445', 'CUST10654', 'CUST10571', 'CUST10614',
'CUST10551',
'CUST10550', 'CUST10700', 'CUST10583', 'CUST10673',
'CUST10067',
'CUST10689', 'CUST10173', 'CUST10942', 'CUST10189',
'CUST10019',
'CUST10417', 'CUST10707', 'CUST10747', 'CUST10050',
'CUST10270',
'CUST10986', 'CUST10607', 'CUST10751', 'CUST10072',
'CUST10537',
'CUST10794', 'CUST10593', 'CUST10243', 'CUST10118',
'CUST10775',
'CUST10021', 'CUST10979', 'CUST10492', 'CUST10177',
'CUST10088',
'CUST10176', 'CUST10181', 'CUST10590', 'CUST10635',
'CUST10594',
'CUST10405', 'CUST10129', 'CUST10473', 'CUST10456',
'CUST10190',
'CUST10725', 'CUST10280', 'CUST10089', 'CUST10784',
'CUST10122',
'CUST10156', 'CUST10756', 'CUST10857', 'CUST10394',
'CUST10109',
'CUST10650', 'CUST10645', 'CUST10023', 'CUST10302',
'CUST10350',
'CUST10856', 'CUST10563', 'CUST10008', 'CUST10738',
'CUST10833',
'CUST10164', 'CUST10254', 'CUST10375', 'CUST10221',
'CUST10278',
'CUST10934', 'CUST10432', 'CUST10169', 'CUST10741',
'CUST10940',
'CUST10104', 'CUST10564', 'CUST10667', 'CUST10061',
'CUST10785',
'CUST10861', 'CUST10910', 'CUST10384', 'CUST10604',
'CUST10500',
'CUST10628', 'CUST10052', 'CUST10297', 'CUST10704',
'CUST10515',
'CUST10163', 'CUST10554', 'CUST10363', 'CUST10810',
'CUST10048',
'CUST10281', 'CUST10844', 'CUST10608', 'CUST10888',
'CUST10973',
'CUST10277', 'CUST10668', 'CUST10703', 'CUST10788',

```
'CUST10343',
      'CUST10649', 'CUST10597', 'CUST10588', 'CUST10729',
      'CUST10548'],
      dtype=object)
```

Ensure all transaction CustomerID

```
# Validate transaction CustomerIDs against customer master data
tran['CustomerID'].isin(cust['CustomerID']).value_counts()
```

```
CustomerID
True      23050
Name: count, dtype: int64
```

Data Cleaning Insight:

No major missing values, duplicate CustomerIDs, or invalid customer references were found after data cleaning. All 23,050 transaction records contain valid CustomerIDs that successfully match with the customer dataset, ensuring consistent, accurate, and reliable data for further analysis.

Step 3: Merge if Needed

- Join Customer_Master_Data with Customer_Transactions on CustomerID (if data is required)

```
#Merge the transactio and Customer datasets on customerID
df=pd.merge(tran,cust,on=['CustomerID'],how="left")
```

```
df.head()
```

	CustomerID	TransactionDate	TransactionAmount	Name \
0	CUST10771	2023-07-31	2383.07	Lakshay Dhillon
1	CUST10100	2024-03-10	497.54	Aniruddh Borah
2	CUST10031	2025-02-17	536.78	Ritvik Ahuja
3	CUST10987	2023-07-17	314.89	Jayan Wagle
4	CUST10831	2024-12-15	2543.19	Ishita Agarwal

	Email	Gender	Age	City
MaritalStatus \				
0	dharmajantara@gmail.com	Female	27	Ahmedabad

Widowed					
1	jivikabhavsar@gmail.com	Female	53	Lucknow	
Divorced					
2	jhaverifarhan@chandra.org	Male	40	Ahmedabad	
Single					
3	ojas82@gmail.com	Not Disclosed	75	Bangalore	
Widowed					
4	vbalay@yahoo.com	Not Disclosed	27	Jaipur	
Divorced					

	NumChildren	JoinDate
0	3	2023-05-31
1	4	2022-01-08
2	0	2023-12-03
3	4	2022-03-18
4	4	2021-06-30

Data Validation Insight:

All transaction CustomerIDs successfully matched with the customer master dataset, confirming complete data consistency.

Step 4: Perform RFM Calculation

● Use groupby on CustomerID for:

o max(TransactionDate) → Recency

```
df.groupby('CustomerID')['TransactionDate'].max()
```

CustomerID	
CUST10000	2025-07-17
CUST10001	2025-06-25
CUST10002	2025-07-12
CUST10003	2025-05-10
CUST10004	2025-07-22
...	...
CUST10995	2024-06-23
CUST10996	2025-07-15
CUST10997	2025-06-28
CUST10998	2025-03-26
CUST10999	2025-07-25

Name: TransactionDate, Length: 1000, dtype: datetime64[ns]

```
#Find the last transaction date for each customer
Lastdate=df.groupby('CustomerID')
['TransactionDate'].max().reset_index().rename(columns={'TransactionDate':'LastTransactionDate'})
```

Lastdate

	CustomerID	LastTransactionDate
0	CUST10000	2025-07-17
1	CUST10001	2025-06-25
2	CUST10002	2025-07-12
3	CUST10003	2025-05-10
4	CUST10004	2025-07-22
...	...	...
995	CUST10995	2024-06-23
996	CUST10996	2025-07-15
997	CUST10997	2025-06-28
998	CUST10998	2025-03-26
999	CUST10999	2025-07-25

[1000 rows x 2 columns]

#calculate the next day after the last transaction date

from datetime import timedelta

today=df['TransactionDate'].max()+timedelta(days=1)

today

Timestamp('2025-07-30 00:00:00')

#Calculate recency for each customer

cust['Recency']=(today-Lastdate['LastTransactionDate']).dt.days

cust['Recency']

0	13
1	35
2	18
3	81
4	8

...	...
995	402
996	15
997	32
998	126
999	5

Name: Recency, Length: 1000, dtype: int64

#Check for missing values in the Recency Column

cust['Recency'].isnull().sum()

np.int64(0)

count(TransactionDate) → Frequency

#Calculate Frequency for each Customer

Frequency=df.groupby('CustomerID')

```
[ 'TransactionDate' ].count().reset_index().rename(columns={ 'Transaction
Date': 'Frequency' })
```

Frequency

	CustomerID	Frequency
0	CUST10000	23
1	CUST10001	30
2	CUST10002	24
3	CUST10003	25
4	CUST10004	19
...	...	...
995	CUST10995	21
996	CUST10996	21
997	CUST10997	20
998	CUST10998	25
999	CUST10999	23

[1000 rows x 2 columns]

sum(TransactionAmount) → Monetary

#Calculate monetary value for each customer

```
Monetary=df.groupby('CustomerID')
```

```
[ 'TransactionAmount' ].sum().reset_index().rename(columns={ 'Transaction
Amount': 'Monetary' })
```

Monetary

	CustomerID	Monetary
0	CUST10000	21265.49
1	CUST10001	28654.31
2	CUST10002	23884.03
3	CUST10003	24206.03
4	CUST10004	25565.30
...	...	...
995	CUST10995	24325.19
996	CUST10996	21809.11
997	CUST10997	21120.48
998	CUST10998	29494.56
999	CUST10999	22028.01

[1000 rows x 2 columns]

```
df.head()
```

	CustomerID	TransactionDate	TransactionAmount	Name \
0	CUST10771	2023-07-31	2383.07	Lakshay Dhillon
1	CUST10100	2024-03-10	497.54	Aniruddh Borah
2	CUST10031	2025-02-17	536.78	Ritvik Ahuja
3	CUST10987	2023-07-17	314.89	Jayan Wagle

4	CUST10831	2024-12-15	2543.19	Ishita Agarwal
		Email	Gender	Age
		City		
MaritalStatus \				
0	dharmajantara@gmail.com	Female	27	Ahmedabad
Widowed				
1	jivikabhavsar@gmail.com	Female	53	Lucknow
Divorced				
2	jhaverifarhan@chandra.org	Male	40	Ahmedabad
Single				
3	ojas82@gmail.com	Not Disclosed	75	Bangalore
Widowed				
4	vbalay@yahoo.com	Not Disclosed	27	Jaipur
Divorced				
	NumChildren	JoinDate		
0	3	2023-05-31		
1	4	2022-01-08		
2	0	2023-12-03		
3	4	2022-03-18		
4	4	2021-06-30		

RFM Analysis Insight: The RFM calculation highlighted diverse customer purchasing patterns, where some customers remained highly active with frequent and recent transactions, while others demonstrated lower engagement due to extended gaps in purchasing activity.

Store result in a new DataFrame called df_rfm

```
# step by step Merge the recency,Frequency,Monetary dataframes on CustomerID
df_rfm=pd.merge(cust[['CustomerID','Recency']],Frequency,on="CustomerID",how="left")

df_rfm=pd.merge(df_rfm,Monetary,on="CustomerID",how="left")

df_rfm.head()
```

	CustomerID	Recency	Frequency	Monetary
0	CUST10000	13	23	21265.49
1	CUST10001	35	30	28654.31
2	CUST10002	18	24	23884.03
3	CUST10003	81	25	24206.03
4	CUST10004	8	19	25565.30

Step 5: Score RFM

- Use quantile-based scoring using `pd.qcut()` or `rank()` and `cut()`
- Create three new columns: `R_Score`, `F_Score`, `M_Score`

```
# Assign R, F, M scores based on quantiles
df_rfm['R_Score']=pd.qcut(df_rfm['Recency'],5,labels=[5,4,3,2,1])
df_rfm['F_Score']=pd.qcut(df_rfm['Frequency'],5,labels=[1,2,3,4,5])
df_rfm['M_Score']=pd.qcut(df_rfm['Monetary'],5,labels=[1,2,3,4,5])

df_rfm.head()
```

	CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score
0	CUST10000	13	23	21265.49	4	3	2
1	CUST10001	35	30	28654.31	3	5	5
2	CUST10002	18	24	23884.03	4	3	3
3	CUST10003	81	25	24206.03	1	4	3
4	CUST10004	8	19	25565.30	5	1	4

RFM Scoring Insight:

High RFM scores identified a small group of highly valuable customers contributing consistently through frequent and high-value purchases

Step 6: Create Combined RFM Segment

Concatenate the R, F, M scores into a string like "555", "432", etc.

```
#Creat RFM Segment by concatenation R,F,M scores
df_rfm['Segment']=df_rfm['R_Score'].astype(str)
+df_rfm['F_Score'].astype(str)+df_rfm['M_Score'].astype(str)

df_rfm.head()
```

	CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	Segment
0	CUST10000	13	23	21265.49	4	3	2	432
1	CUST10001	35	30	28654.31	3	5	5	355
2	CUST10002	18	24	23884.03	4	3	3	433
3	CUST10003	81	25	24206.03	1	4	3	

143						
4	CUST10004	8	19	25565.30	5	1
514						4

Step 7: Assign Segment Labels

- Use business rules to define segment labels for selected score combinations

- Example:

o RFM 555 → Champion o RFM 111 → Lost

```
#step by step Define segment labels based on RFM segments
df_rfm['Segment_Labels']='others'

df_rfm.loc[df_rfm['Segment']=='555','Segment_Labels']='Champions'
df_rfm.loc[df_rfm['Segment']=='111','Segment_Labels']='Lost'

df_rfm.loc[(df_rfm['Segment'].str[0]=='5')&(df_rfm['Segment']!=
='555'),'Segment_Labels']='Recent'

df_rfm.loc[(df_rfm['Segment'].str[1]=='5')&(df_rfm['Segment']!=
='555'),'Segment_Labels']='Frequent'

df_rfm.loc[(df_rfm['Segment'].str[2]=='5')&(df_rfm['Segment']!=
='555'),'Segment_Labels']='Big Spender'

#Analyze the distribution of customers access segments
df_rfm['Segment_Labels'].value_counts()

Segment_Labels
others      543
Big Spender 165
Recent      160
Frequent     63
Champions    35
Lost         34
Name: count, dtype: int64

# Analyze the distribution of customers across segments
labels=df_rfm['Segment_Labels'].value_counts().values
index=df_rfm['Segment_Labels'].value_counts().index
index

Index(['others', 'Big Spender', 'Recent', 'Frequent', 'Champions',
'Lost'], dtype='object', name='Segment_Labels')
```

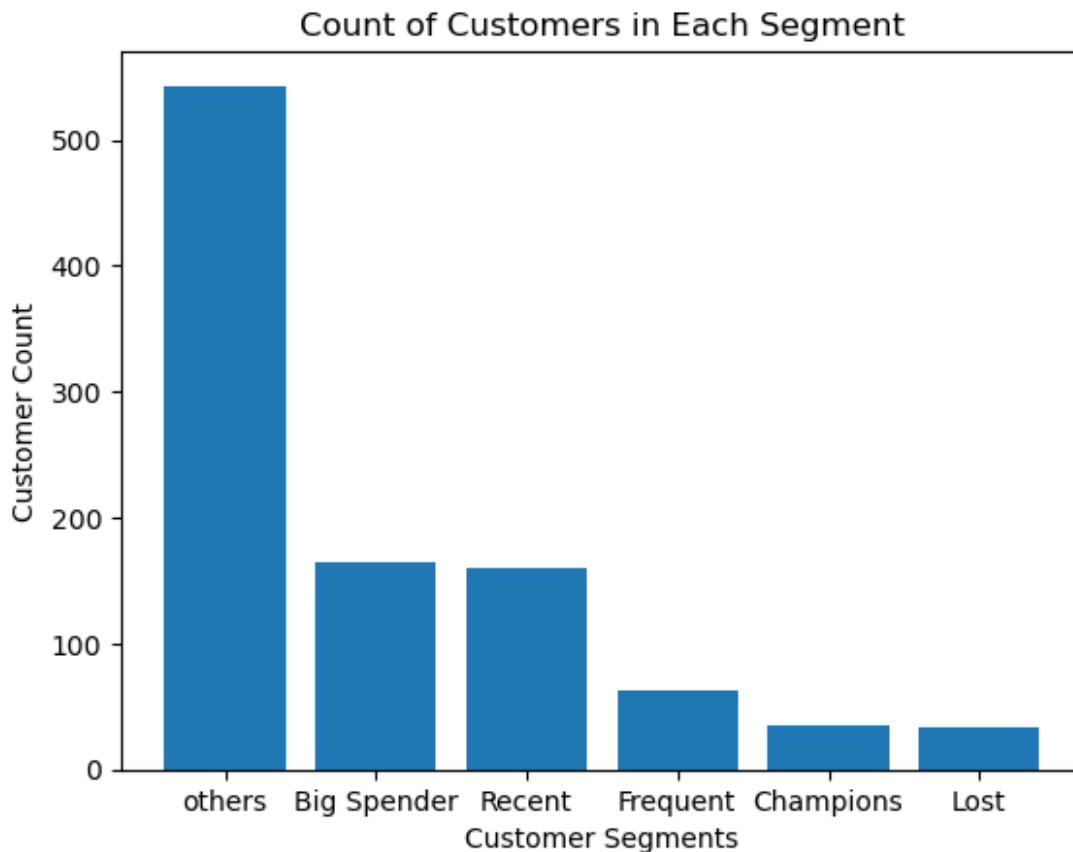
RFM Segment Insight:

The largest customer group belonged to the “Others” segment (543 customers), indicating a broad mix of average purchasing behavior

Step 8: Visualize

Count of customers in each segment

```
import matplotlib.pyplot as plt
segment_counts = df_rfm['Segment_Labels'].value_counts()
plt.bar(segment_counts.index, segment_counts.values)
plt.xlabel('Customer Segments')
plt.ylabel('Customer Count')
plt.title('Count of Customers in Each Segment')
plt.show()
```



Big Spender Insight :

The “Big Spender” segment contains 165 customers and contributes 22.3% of total revenue, making it one of the most valuable customer groups despite its relatively smaller size.

Champion Segment Insight

Only 35 customers were classified as “Champions,” but they generated high transaction values with very recent purchasing activity. This segment represents the most valuable and loyal customers of the business.

Lost Customer Insight

A total of 34 customers were classified as “Lost,” indicating low purchase frequency, low spending behavior, and long periods of inactivity. These customers may require re-engagement strategies to reduce churn.

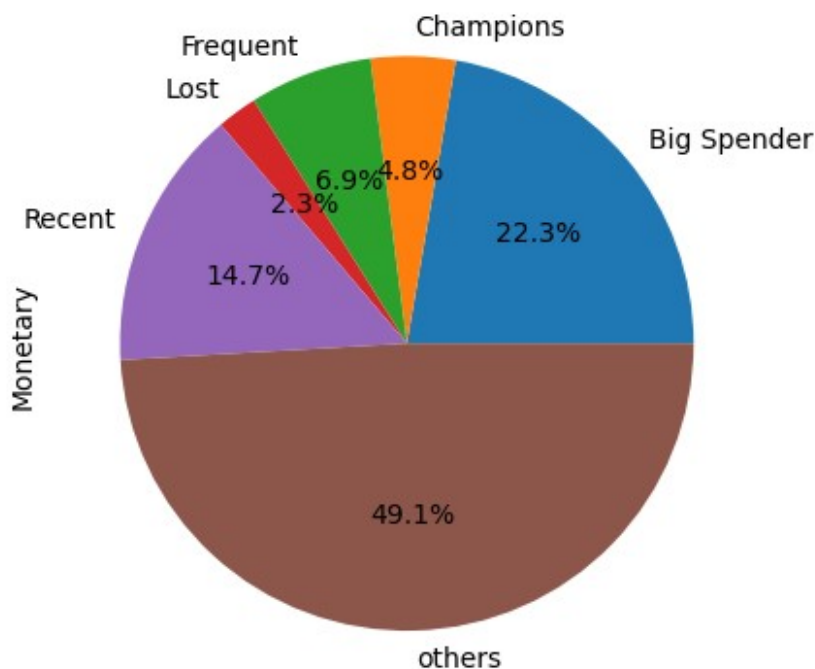
Recent Customer Insight

Around 160 customers were categorized as “Recent,” showing that many customers made purchases recently. This indicates strong current engagement and potential opportunities for retention and repeat purchases.

● Revenue contribution per segment

```
df_rfm.groupby('Segment_Labels')  
['Monetary'].sum().plot(kind='pie', autopct="%0.1f%%")
```

<Axes: ylabel='Monetary'>



Big Spender customers account for 22.3% of total revenue, the second-highest among all segments, despite containing only 165 customers. This indicates that these customers have high purchasing power and contribute significantly to business revenue.

● Recency vs Monetary scatter plot colored by segment

```
df_rfm.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            1000 non-null   object
1   Recency                1000 non-null   int64
2   Frequency              1000 non-null   int64
3   Monetary               1000 non-null   float64
4   R_Score                1000 non-null   category
5   F_Score                1000 non-null   category
6   M_Score                1000 non-null   category
7   Segment                1000 non-null   object
8   Segment_Labels        1000 non-null   object
dtypes: category(3), float64(1), int64(2), object(3)
memory usage: 50.6+ KB

df_rfm['Segment_Labels'].value_counts()

Segment_Labels
others          543
Big Spender     165
Recent          160
Frequent        63
Champions       35
Lost            34
Name: count, dtype: int64

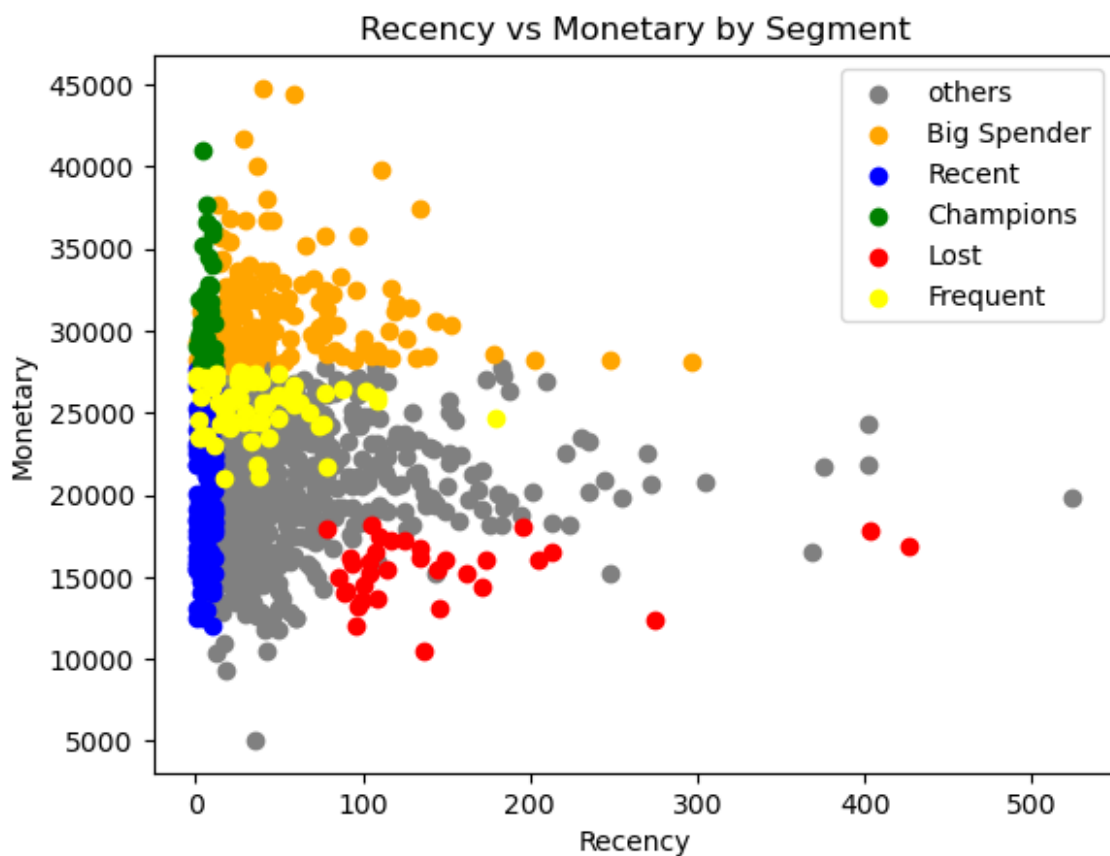
palette = {
    'others': 'gray',
    'Big Spender': 'orange',
    'Recent': 'blue',
    'Frequent': 'yellow',
    'Lost': 'red',
    'Champions': 'green'
}

plt.figure()

for segment in df_rfm['Segment_Labels'].unique():
    subset = df_rfm[df_rfm['Segment_Labels'] == segment]
```

```
plt.scatter(
    subset['Recency'],
    subset['Monetary'],
    label=segment,
    color=palette.get(segment)
)

plt.xlabel("Recency")
plt.ylabel("Monetary")
plt.title("Recency vs Monetary by Segment")
plt.legend()
plt.show()
```



The scatter plot shows that customers with lower recency values (customers who purchased recently) tend to have higher monetary spending. A large concentration of points is visible between recency 0–30 days, indicating that recently active customers make frequent and higher-value purchases. On the other hand, customers with very high recency values are fewer in number and generally show lower spending behavior. This suggests a negative relationship between recency and monetary value — as recency increases, customer spending tends to decrease. The segmented scatter plot further shows that “Big Spender” and “Champion” customers are mostly concentrated in the low-recency and high-monetary region, whereas “Lost” customers are spread across higher recency values with comparatively lower spending.

Pareto Analysis: Show how top 20% customers contribute to 80% revenue

```
df=df_rfm[['CustomerID','Monetary']]
df=df.sort_values(by='Monetary',ascending=False)
df['Cumulative_Revenue']=df['Monetary'].cumsum()
df['Revenue_percent']=df['Cumulative_Revenue']/
df['Monetary'].sum()*100
df=df.reset_index(drop=True)
df['Customer_percent']=(df.index+1)/len(df)*100
df[df['Customer_percent']<=20].tail(1)
```

	CustomerID	Monetary	Cumulative_Revenue	Revenue_percent	\
199	CUST10338	27838.98	6231163.35	27.029495	
	Customer_percent				
199		20.0			

Based on the Pareto Principle, the top 20% of customers should contribute 80% of the revenue. However, this condition is not satisfied in the dataset. Hence, the data does not satisfy the Pareto rule.

```
df[df['Customer_percent']<=50].tail(1)
```

	CustomerID	Monetary	Cumulative_Revenue	Revenue_percent	\
499	CUST10516	22987.87	13785960.82	59.800639	
	Customer_percent				
499		50.0			

```
df[df['Customer_percent']<=70].tail(1)
```

	CustomerID	Monetary	Cumulative_Revenue	Revenue_percent	\
699	CUST10191	19792.84	18055448.07	78.32079	
	Customer_percent				
699		70.0			

```
df[df['Customer_percent']<=80].tail(1)
```

	CustomerID	Monetary	Cumulative_Revenue	Revenue_percent	\
799	CUST10289	18164.57	19947157.43	86.526633	
	Customer_percent				
799		80.0			

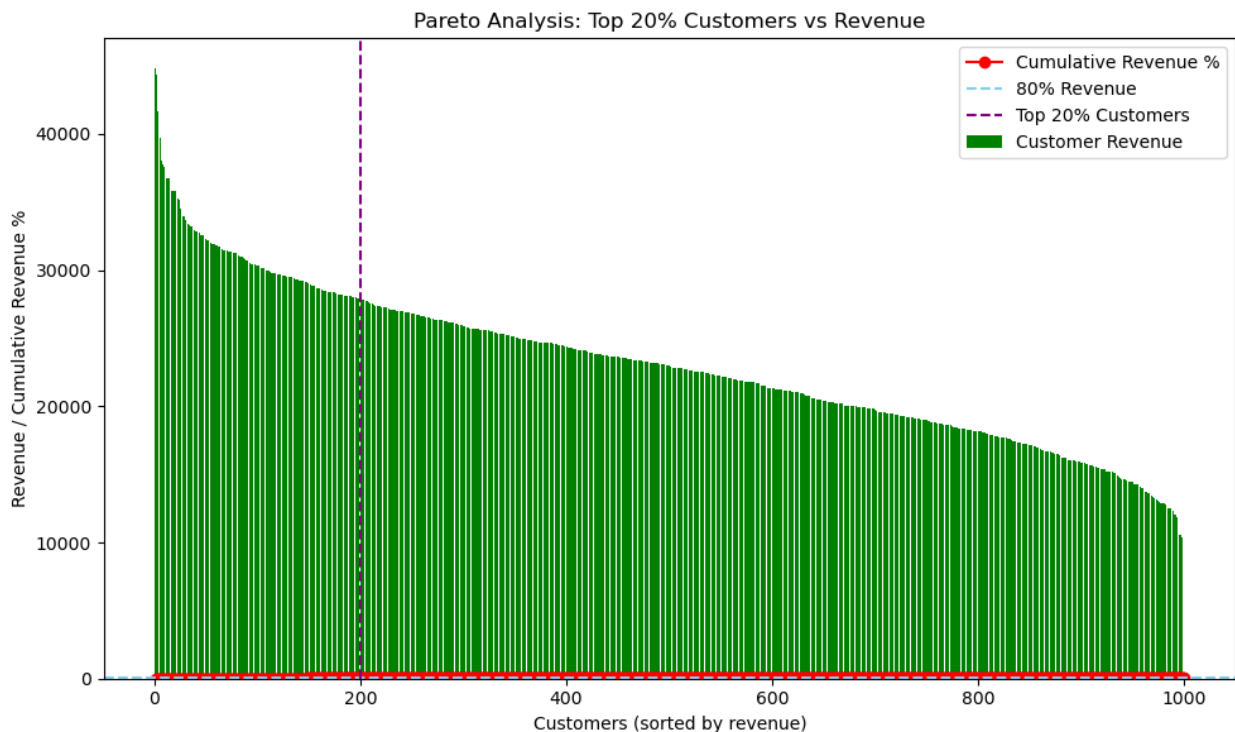
```

plt.figure(figsize=(10, 6))
plt.bar(df.index + 1, df['Monetary'], color='green', label='Customer Revenue')
plt.plot(df.index + 1, df['Revenue_percent'], color='red', marker='o', label='Cumulative Revenue %')

# Reference lines
plt.axhline(80, color='skyblue', linestyle='--', label='80% Revenue')
plt.axvline(len(df) * 0.2, color='purple', linestyle='--', label='Top 20% Customers')

plt.title('Pareto Analysis: Top 20% Customers vs Revenue')
plt.xlabel('Customers (sorted by revenue)')
plt.ylabel('Revenue / Cumulative Revenue %')
plt.legend()
plt.tight_layout()
plt.show()

```



The graph indicates that the dataset does not follow the Pareto (80/20) rule, as the top 20% of customers do not contribute 80% of the total revenue. Instead, revenue is distributed more evenly across customers, showing that the business depends on a broader customer base rather than a small group of high-revenue customers.